

D1. Provide screenshots to show the status of all packages loaded onto each truck at a time between 8:35 a.m. and 9:25 a.m.

```
[nix-shell:~/Documents/WGU/C950-v6]$ python main.py
Truck 1 return time: 10:45:20
Truck 2 return time: 11:28:40
Truck 3 return time: 12:16:20

Simulation complete. Total mileage: 120.0

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): 2
Enter time to check (HH:MM, 24-hour format): 09:24

--- All Package Statuses at 09:24:00 ---
ID: 01 | Status: delivered at 08:39:00 | Address: 195 W Oakland Ave
ID: 02 | Status: at hub | Address: 2530 S 500 E
ID: 03 | Status: en route | Address: 233 Canyon Rd
ID: 04 | Status: delivered at 09:17:00 | Address: 380 W 2880 S
ID: 05 | Status: delivered at 09:03:20 | Address: 410 S State St
ID: 06 | Status: en route | Address: 3060 Lester St
ID: 07 | Status: delivered at 08:29:40 | Address: 1330 2100 S
ID: 08 | Status: delivered at 09:06:40 | Address: 300 State St
ID: 09 | Status: at hub | Address: 300 State St
ID: 10 | Status: delivered at 09:16:00 | Address: 600 E 900 South
ID: 11 | Status: en route | Address: 2600 Taylorsville Blvd
ID: 12 | Status: en route | Address: 3575 W Valley Central Station bus Loop
ID: 13 | Status: en route | Address: 2010 W 500 S
ID: 14 | Status: delivered at 08:06:20 | Address: 4300 S 1300 E
ID: 15 | Status: delivered at 08:13:00 | Address: 4580 S 2300 E
ID: 16 | Status: delivered at 08:13:00 | Address: 4580 S 2300 E
ID: 17 | Status: at hub | Address: 3148 S 1100 W
ID: 18 | Status: en route | Address: 1488 4800 S
ID: 19 | Status: delivered at 08:45:20 | Address: 177 W Price Ave
ID: 20 | Status: delivered at 08:47:00 | Address: 3595 Main St
ID: 21 | Status: delivered at 09:11:40 | Address: 3595 Main St
ID: 22 | Status: en route | Address: 6351 South 900 East
ID: 23 | Status: en route | Address: 5100 South 2700 West
ID: 24 | Status: en route | Address: 5025 State St
ID: 25 | Status: en route | Address: 5383 South 900 East #104
ID: 26 | Status: at hub | Address: 5383 South 900 East #104
ID: 27 | Status: at hub | Address: 1060 Dalton Ave S
ID: 28 | Status: at hub | Address: 2835 Main St
ID: 29 | Status: delivered at 08:29:40 | Address: 1330 2100 S
ID: 30 | Status: delivered at 09:06:40 | Address: 300 State St
ID: 31 | Status: delivered at 09:22:40 | Address: 3365 S 900 W
ID: 32 | Status: at hub | Address: 3365 S 900 W
ID: 33 | Status: at hub | Address: 2530 S 500 E
ID: 34 | Status: en route | Address: 4580 S 2300 E
ID: 35 | Status: at hub | Address: 1060 Dalton Ave S
ID: 36 | Status: en route | Address: 2300 Parkway Blvd
ID: 37 | Status: delivered at 09:03:20 | Address: 410 S State St
ID: 38 | Status: en route | Address: 410 S State St
ID: 39 | Status: at hub | Address: 2010 W 500 S
ID: 40 | Status: delivered at 09:17:00 | Address: 380 W 2880 S

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): █
```

D2. Provide screenshots to show the status of all packages loaded onto each truck at a time between 9:35 a.m. and 10:25 a.m.

```
[nix-shell:~/Documents/WGU/C950-v6]$ python main.py
Truck 1 return time: 10:45:20
Truck 2 return time: 11:28:40
Truck 3 return time: 12:16:20

Simulation complete. Total mileage: 120.0

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): 2
Enter time to check (HH:MM, 24-hour format): 10:24

--- All Package Statuses at 10:24:00 ---

ID: 01 | Status: delivered at 08:39:00 | Address: 195 W Oakland Ave
ID: 02 | Status: at hub | Address: 2530 S 500 E
ID: 03 | Status: en route | Address: 233 Canyon Rd
ID: 04 | Status: delivered at 09:17:00 | Address: 380 W 2880 S
ID: 05 | Status: delivered at 09:03:20 | Address: 410 S State St
ID: 06 | Status: delivered at 09:27:40 | Address: 3060 Lester St
ID: 07 | Status: delivered at 08:29:40 | Address: 1330 2100 S
ID: 08 | Status: delivered at 09:06:40 | Address: 300 State St
ID: 09 | Status: at hub | Address: 410 S State St
ID: 10 | Status: delivered at 09:16:00 | Address: 600 E 900 South
ID: 11 | Status: delivered at 10:24:00 | Address: 2600 Taylorsville Blvd
ID: 12 | Status: delivered at 09:57:00 | Address: 3575 W Valley Central Station bus Loop
ID: 13 | Status: delivered at 09:33:00 | Address: 2010 W 500 S
ID: 14 | Status: delivered at 08:06:20 | Address: 4300 S 1300 E
ID: 15 | Status: delivered at 08:13:00 | Address: 4580 S 2300 E
ID: 16 | Status: delivered at 08:13:00 | Address: 4580 S 2300 E
ID: 17 | Status: at hub | Address: 3148 S 1100 W
ID: 18 | Status: delivered at 09:46:20 | Address: 1488 4800 S
ID: 19 | Status: delivered at 08:45:20 | Address: 177 W Price Ave
ID: 20 | Status: delivered at 08:47:00 | Address: 3595 Main St
ID: 21 | Status: delivered at 09:11:40 | Address: 3595 Main St
ID: 22 | Status: delivered at 10:13:20 | Address: 6351 South 900 East
ID: 23 | Status: delivered at 09:48:20 | Address: 5100 South 2700 West
ID: 24 | Status: delivered at 10:03:20 | Address: 5025 State St
ID: 25 | Status: delivered at 10:09:00 | Address: 5383 South 900 East #104
ID: 26 | Status: at hub | Address: 5383 South 900 East #104
ID: 27 | Status: at hub | Address: 1060 Dalton Ave S
ID: 28 | Status: at hub | Address: 2835 Main St
ID: 29 | Status: delivered at 08:29:40 | Address: 1330 2100 S
ID: 30 | Status: delivered at 09:06:40 | Address: 300 State St
ID: 31 | Status: delivered at 09:22:40 | Address: 3365 S 900 W
ID: 32 | Status: at hub | Address: 3365 S 900 W
ID: 33 | Status: at hub | Address: 2530 S 500 E
ID: 34 | Status: en route | Address: 4580 S 2300 E
ID: 35 | Status: at hub | Address: 1060 Dalton Ave S
ID: 36 | Status: delivered at 09:33:00 | Address: 2300 Parkway Blvd
ID: 37 | Status: delivered at 09:03:20 | Address: 410 S State St
ID: 38 | Status: en route | Address: 410 S State St
ID: 39 | Status: at hub | Address: 2010 W 500 S
ID: 40 | Status: delivered at 09:17:00 | Address: 380 W 2880 S

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): █
```

D3. Provide screenshots to show the status of all packages loaded onto each truck at a time between 12:03 p.m. and 1:12 p.m.

```
[nix-shell:~/Documents/WGU/C950-v6]$ python main.py
Truck 1 return time: 10:45:20
Truck 2 return time: 11:28:40
Truck 3 return time: 12:16:20

Simulation complete. Total mileage: 120.0

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): 2
Enter time to check (HH:MM, 24-hour format): 13:11

--- All Package Statuses at 13:11:00 ---

ID: 01 | Status: delivered at 08:39:00 | Address: 195 W Oakland Ave
ID: 02 | Status: delivered at 11:09:20 | Address: 2530 S 500 E
ID: 03 | Status: delivered at 11:03:20 | Address: 233 Canyon Rd
ID: 04 | Status: delivered at 09:17:00 | Address: 380 W 2880 S
ID: 05 | Status: delivered at 09:03:20 | Address: 410 S State St
ID: 06 | Status: delivered at 09:27:40 | Address: 3060 Lester St
ID: 07 | Status: delivered at 08:29:40 | Address: 1330 2100 S
ID: 08 | Status: delivered at 09:06:40 | Address: 300 State St
ID: 09 | Status: delivered at 11:54:40 | Address: 410 S State St
ID: 10 | Status: delivered at 09:16:00 | Address: 600 E 900 South
ID: 11 | Status: delivered at 10:24:00 | Address: 2600 Taylorsville Blvd
ID: 12 | Status: delivered at 09:57:00 | Address: 3575 W Valley Central Station bus Loop
ID: 13 | Status: delivered at 09:33:00 | Address: 2010 W 500 S
ID: 14 | Status: delivered at 08:06:20 | Address: 4300 S 1300 E
ID: 15 | Status: delivered at 08:13:00 | Address: 4580 S 2300 E
ID: 16 | Status: delivered at 08:13:00 | Address: 4580 S 2300 E
ID: 17 | Status: delivered at 11:21:40 | Address: 3148 S 1100 W
ID: 18 | Status: delivered at 09:46:20 | Address: 1488 4800 S
ID: 19 | Status: delivered at 08:45:20 | Address: 177 W Price Ave
ID: 20 | Status: delivered at 08:47:00 | Address: 3595 Main St
ID: 21 | Status: delivered at 09:11:40 | Address: 3595 Main St
ID: 22 | Status: delivered at 10:13:20 | Address: 6351 South 900 East
ID: 23 | Status: delivered at 09:48:20 | Address: 5100 South 2700 West
ID: 24 | Status: delivered at 10:03:20 | Address: 5025 State St
ID: 25 | Status: delivered at 10:09:00 | Address: 5383 South 900 East #104
ID: 26 | Status: delivered at 10:53:20 | Address: 5383 South 900 East #104
ID: 27 | Status: delivered at 11:38:40 | Address: 1060 Dalton Ave S
ID: 28 | Status: delivered at 11:13:00 | Address: 2835 Main St
ID: 29 | Status: delivered at 08:29:40 | Address: 1330 2100 S
ID: 30 | Status: delivered at 09:06:40 | Address: 300 State St
ID: 31 | Status: delivered at 09:22:40 | Address: 3365 S 900 W
ID: 32 | Status: delivered at 11:23:40 | Address: 3365 S 900 W
ID: 33 | Status: delivered at 11:09:20 | Address: 2530 S 500 E
ID: 34 | Status: delivered at 10:29:00 | Address: 4580 S 2300 E
ID: 35 | Status: delivered at 11:38:40 | Address: 1060 Dalton Ave S
ID: 36 | Status: delivered at 09:33:00 | Address: 2300 Parkway Blvd
ID: 37 | Status: delivered at 09:03:20 | Address: 410 S State St
ID: 38 | Status: delivered at 11:00:00 | Address: 410 S State St
ID: 39 | Status: delivered at 11:44:00 | Address: 2010 W 500 S
ID: 40 | Status: delivered at 09:17:00 | Address: 380 W 2880 S

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): 
```

E. Provide screenshots showing successful completion of the code that includes the total mileage traveled by *all* trucks.

```
[nix-shell:~/Documents/WGU/C950-v6]$ python main.py
Truck 1 return time: 10:45:20
Truck 2 return time: 11:28:40
Truck 3 return time: 12:16:20

Simulation complete. Total mileage: 120.0

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): 3
Total mileage traveled by all trucks: 120.0 miles

=====
WGUPS Package Tracking System
=====
1 - Look up a package by ID
2 - Show all package statuses at a given time
3 - Display total mileage for all trucks
4 - Exit
=====
Enter your choice (1-4): █
```

F. Algorithm Justification

F1. Describe two or more strengths of the algorithm used in the solution.

The solution uses a Nearest-Neighbor Greedy Algorithm, which selects the next closest undelivered package location at each stop. This algorithm has several notable strengths.

Strength 1 - Simplicity: The nearest-neighbor algorithm is very simple, making it easy to understand and implement for small projects. Simplicity and ease of understanding are

the foundation of predictability and reliability, which makes the nearest-neighbor algorithm still useful and popular despite its $O(n^2)$ computational complexity.

Strength 2 - Consistency: Nearest-neighbor algorithms don't always find the perfect route, but they reliably find a good one. Studies show the routes they produce are typically only about 15% longer than the best possible route (P, 2022). In this case, all 40 packages were delivered with a total mileage of 120 miles, which is well under the 140 mile limit, and every deadline was met.

Strength 3 - Speed and Efficiency: Greedy algorithms do not revisit previously made choices, making them fast and practical for routing problems where a good solution is needed quickly rather than a perfect one (Codecademy, n.d.).

F2. Verify that the algorithm used in the solution meets all requirements in the scenario.

All 40 packages delivered: Nearest-neighbor iterates through all packages on each truck until the list is empty. The simulation confirms that all 40 packages are delivered.

Total mileage $\leq$ 140 miles: Combined mileage: Truck 1 (49.6) + Truck 2 (43.1) + Truck 3 (27.3) = 120 miles.

9:00 AM deadline (package 15): Package 15 is loaded on Truck 1 departing at 8:00 AM and delivered at 8:13 AM.

10:30 AM deadlines: All 10:30 AM packages are distributed across Truck 1 and Truck 2. All deliveries are made before 10:30 AM.

Truck 2 only packages (3, 18, 36, 38): These packages are explicitly placed in the Truck 2 manifest before the simulation runs.

Delayed packages (6, 25, 28, 32): Packages 6 and 25 are assigned to Truck 2. Truck 2's departure time is 9:05 AM. Packages 28 and 32 are assigned to Truck 3. Truck 3's departure time is 10:45 AM.

Wrong address for package 9: Package 9 is loaded on Truck 3. Truck 3's departure time is 10:45 AM. The address update function is called before Truck 3 departs.

Co-delivery group, packages 13, 14, 15, 16, 19, 20: All six packages are placed on Truck 1 and delivered on the same route.

Max 16 packages per truck: Truck 1: 16 packages, Truck 2: 14 packages, Truck 3: 10 packages. No truck exceeds 16.

Two drivers: Trucks 1 and 2 run concurrently (Driver 1 and Driver 2). Driver 1 returns to the hub at 10:45 AM and then operates Truck 3. Only 2 trucks operate at any one time.

F3. Identify two other named algorithms that are different from the algorithm implemented in the solution and would meet all requirements in the scenario.

Algorithm 1 - Dijkstra's Shortest Path Algorithm: Dijkstra's algorithm finds the shortest path between two points in a network (DSA Dijkstra's Algorithm). It works by maintaining a priority queue of unvisited nodes and greedily selecting the nearest node. For this routing problem, it could be used to calculate the shortest distance between every pair of delivery addresses and the hub before the trucks even leave. This would decrease route distances compared to the naïve nearest-neighbor distance algorithm, which uses straight-line table distances rather than pre-computed distances.

Algorithm 2 - Greedy Algorithm with 2-opt Local Search: The 2-opt algorithm starts with a random working route and repeatedly tries to improve it (Stephens, n.d.). It does this by selecting two stops on the route and swapping their order to see if the total distance gets shorter. It keeps doing this over and over again until no more improvements can be found.

F3A. Describe how *both* algorithms identified in part F3 are different from the algorithm used in the solution.

The nearest-neighbor algorithm and Dijkstra's algorithm differ in how they find distances. Nearest-neighbor just chooses the closest next stop using the distance table. Dijkstra's does more work upfront by calculating the shortest path between every pair of locations before the delivery begins, which generally produces better results but takes more time, complexity, and computational power.

The nearest-neighbor algorithm and the 2-opt method differ in the number of passes they make. Nearest-neighbor chooses once per stop, no second looks. The 2-opt method is similar but differs in that it checks whether it can improve the route by swapping pairs of delivery locations until no better option is available. This makes 2-opt more accurate, yet slower and more computationally expensive.

G. Describe what you would do differently, other than the two algorithms identified in part F3, if you did this project again, including details of the modifications that would be made.

If I were to do this project again, I would modify the truck loading to be automated instead of hardcoded, as in my current project. In the current project, package IDs are manually assigned to each truck in the main code. A more ambitious strategy would have the algorithm account for the special notes from the package file, placing the packages as required into each truck. Second, more time would be spent fine-tuning the route to produce a lower total mileage. The nearest-neighbor algorithm produces a good mileage below the requirement, but not the shortest possible. Running additional tests and adjusting package assignments could reduce the total mileage.

H. Verify that the data structure used in the solution meets all requirements in the scenario.

The solution uses a chaining hash table with a capacity set to 40 buckets, one per package. Each bucket is a list of key-value pairs. The hash table runs with a complexity of $O(1)$ in the average case per insert and lookup for integer keys.

The hash table satisfies all requirements:

- No external libraries are used.
- All 8 package data fields are stored in Package objects and inserted into the table.
- The insert method takes the package_id as the key and computes its bucket via $\text{key} \% \text{capacity}$. It adds to the list or overwrites the existing package.
- The lookup method takes the package_id, computes the bucket index, and returns the matching package object or None if none is found.

H1 and H1A. Identify two other data structures that could meet the same requirements in the scenario. Describe how *each* data structure identified in H1 is different from the data structure used in the solution.

Data Structure 1 - Python List (Dictionary): Since package IDs run from 1 to 40, a plain Python list could store all packages by using the package ID as the index. Looking up a package would be instant since no hashing is needed. **The difference** from the hash table is that a list only works well when IDs are short, consecutive numbers. The hash table is more flexible and works with any type of key, not just integers in a small range.

Data Structure 2 - Binary Search Tree (BST): A BST stores packages in sorted order by ID (GeeksforGeeks, 2023). Lookup works by repeatedly cutting the remaining options in half

until the correct package is found. This is slightly slower than a hash table for lookups but has the advantage of being able to return packages in sorted order without any extra work. **The difference** from a hash table is that a BST is more complex to build, while the hash table is simpler to implement and faster for straightforward lookups.

I. Sources

Codecademy. (n.d.). Greedy algorithms: Concept, examples, and applications. Retrieved June 2, 2026, from <https://www.codecademy.com/article/greedy-algorithm-explained>

GeeksforGeeks. (2023, September 11). Tree sort. Retrieved June 2, 2026, from <https://www.geeksforgeeks.org/dsa/tree-sort/>

P, Sindhukumari. (2022, December 25). Traveling salesman problem. Medium. Retrieved June 2, 2026, from <https://sindhukumari.medium.com/traveling-salesman-problem-9deb6853ac6>

Stephens, Rod. (n.d.). Understand the 2-opt algorithm for solving the traveling salesman problem. Rod Stephens Books. Retrieved June 2, 2026, from https://rodstephensbooks.com/tsp_2opt.html

W3Schools. (n.d.). DSA Dijkstra's algorithm. Retrieved June 2, 2026, from https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php